

Chapter 1: Introduction

1.1 Background

Modern email systems generate and receive a large number of messages every day, including academic communication, work-related messages, job and internship opportunities, meetings, notifications, promotions, and personal emails. As the number of emails increases, users may find it difficult to identify important messages quickly and understand lengthy email content.

Natural Language Processing (NLP) provides techniques for automatically processing and understanding human language. In email management, NLP can be used to analyze email content, determine its priority, classify it into relevant categories, and generate concise summaries.

IRIS – An Intelligent Email Analysis and Summarization System is being developed as an intelligent email management application that combines NLP-based email analysis with additional smart email-management features. The system is designed to provide **priority classification, category classification, email summarization, smart reply suggestions, email search, behaviour-based personalization, reminders, opportunity detection, explainable AI, and smart email management**.

The current academic implementation focuses primarily on the core NLP tasks of **priority classification, category classification, and email summarization**. These components have been implemented and evaluated using appropriate NLP and machine learning techniques, while the complete application and additional intelligent features are currently under development.

The project demonstrates how NLP and machine learning can be applied to help users understand, organize, and manage their emails more efficiently.

1.2 Problem Statement

Users often receive emails with different levels of importance and different types of information. Manually reading every email, identifying its importance, understanding its main content, and organizing it can be time-consuming, particularly when the number of emails is large.

Existing email management generally requires users to **manually identify important messages, read lengthy content, organize emails, and remember emails that require future action**. This creates a need for an intelligent system that can assist users in analyzing and managing their emails automatically.

The problem addressed by IRIS is to develop an **intelligent email analysis and management system** that can:

- Automatically determine the **priority** of an email as HIGH, MEDIUM, or LOW.
- Classify emails into relevant **categories**.

- Generate a concise **summary** of email content.
- Help users quickly identify important information from their inbox.
- Provide intelligent assistance through features such as **smart replies, search, reminders, behaviour-based organization, and opportunity detection**.
- Support smarter email management while considering **privacy and explainability**.

1.3 Motivation

The motivation behind IRIS is to **reduce the effort required to manage a large number of emails**. Important messages such as interview confirmations, deadlines, academic notices, project updates, and urgent requests may require immediate attention, while less important messages can be reviewed later.

Reading every email completely can also be inefficient when the user only needs to understand its main purpose. An automatic summarization system can provide a concise representation of the important information in an email.

Therefore, IRIS combines **priority classification, category classification, and summarization** with additional intelligent email-management features. The goal is to help users understand their emails faster and make email management more organized and efficient.

1.4 Objectives

The main objectives of the IRIS project are:

1. To preprocess email text using NLP techniques such as **lowercasing, tokenization, stop-word removal, and lemmatization**.
2. To represent email text using **TF-IDF with unigram and bigram features**.
3. To classify emails into **HIGH, MEDIUM, and LOW priority**.
4. To classify emails into relevant **email categories**.
5. To compare different machine learning approaches for email classification.
6. To generate **extractive summaries** of emails using TF-IDF-based sentence scoring.
7. To evaluate classification performance using **accuracy, precision, recall, F1-score, and confusion matrices**.
8. To evaluate the summarization component using **ROUGE metrics**.
9. To integrate the NLP components into a working backend system.
10. To develop intelligent email-management features such as **smart reply suggestions, email search, reminders, and personalized email organization**.
11. To explore **explainable and privacy-focused AI** for email processing.
12. To provide a unified system for **email analysis and intelligent email management**.

1.5 Scope

The scope of IRIS extends beyond basic email classification and summarization. The application is being developed as an intelligent email management system that can analyze emails and assist users in organizing and handling them.

The major components of the project include:

Core NLP Components

- Email priority classification
- Email category classification
- Extractive email summarization
- Text preprocessing
- TF-IDF feature extraction
- Unigram and bigram analysis
- Machine learning-based classification

Intelligent Email-Management Features

- Smart reply suggestions
- Email search
- Behaviour-based personalization
- Smart email cleanup with user approval
- Trash recovery
- Important-email reminders
- Job/opportunity detection
- Explainable AI
- Intelligent email organization

Privacy and Security

The application is also designed with privacy and security considerations, including secure authentication, protected email access, and privacy-focused processing. More advanced privacy mechanisms can be extended as the application develops.

The **current academic evaluation focuses on priority classification, category classification, and email summarization**, while several of the additional intelligent email-management features are still under development.

1.6 Applications

IRIS can be useful in scenarios where users receive large numbers of emails, including:

- **Academic communication** – identifying important notices, assignments, deadlines, and college communication.
- **Job and internship communication** – highlighting interview confirmations, applications, recruitment updates, and opportunities.
- **Work and project communication** – identifying important project updates, meetings, and deadlines.
- **Personal email management** – organizing and summarizing personal messages.
- **Reminder management** – identifying emails that may require future action.
- **Opportunity detection** – identifying potentially relevant job, internship, event, or other opportunities.
- **General inbox management** – helping users understand, prioritize, categorize, and manage incoming emails.
- **Smart email assistance** – supporting users with features such as search and smart reply suggestions.

Review of Existing Research

2.1 Introduction

Email management has been studied from several perspectives, including email importance classification, task and intent detection, content-based classification, machine-learning-based prioritization, and efficient importance labeling. These studies provide the foundation for developing intelligent email systems that can automatically analyze email content and identify information requiring greater attention.

The literature reviewed for this project covers both traditional machine-learning approaches and more recent approaches to email importance labeling. The reviewed studies also demonstrate that email analysis can use different types of information, including lexical features, email metadata, task information, and network-based features.

2.2 Review of Existing Research

2.2.1 Zhang and Xu (2015) — Annotation and Classification of an Email Importance Corpus

Zhang and Xu studied **email importance classification** and created an annotated corpus specifically for this purpose. Their work treats email importance as a classification problem, providing a foundation for systems that automatically determine which emails deserve greater attention. This work is particularly relevant to IRIS because IRIS also treats email importance as a classification task. However, IRIS uses a simplified **HIGH, MEDIUM, and LOW priority scheme** and evaluates **Multinomial Naive Bayes and Logistic Regression** models.

2.2.2 Sappelli et al. (2016) — Assessing E-mail Intent and Tasks in E-mail Messages

Sappelli et al. studied how **tasks and intentions are expressed in corporate email messages**. The researchers developed a task taxonomy and manually annotated the **Enron and Avocado email datasets**. They also investigated automatic classification of email tasks using machine-learning techniques. Their study found that detecting tasks, reply expectations, and time sensitivity can contribute to a more detailed estimation of email priority. This is relevant to IRIS because information such as **requests, actions, deadlines, and tasks** can influence the priority assigned to an email.

2.2.3 Alkhereyf and Rambow (2017) — Work Hard, Play Hard: Email Classification on the Avocado and Enron Corpora

Alkhereyf and Rambow investigated email classification using the **Enron and Avocado corpora**. Their study compared lexical information obtained from email content with information derived from email-exchange networks. They evaluated **SVM and Extra-Trees** classifiers and found that combining lexical and network features improved classification performance for their business-versus-personal classification task. This demonstrates that email classification can benefit from features beyond the textual content itself. In contrast, IRIS focuses on **lightweight text-based features, particularly TF-IDF**, making the approach simpler and more suitable for local deployment.

2.2.4 Zajic, Dorr and Lin (2008) — Single-document and Multi-document Summarization Techniques for Email Threads

Zajic, Dorr, and Lin investigated **email-thread summarization** using two approaches: Individual Message Summarization (IMS) and Collective Message Summarization (CMS). Their experiments were conducted using the **Enron email collection**. The authors used linguistic and statistical methods to generate sentence compressions and create summaries. Their results showed that the collective message approach was more effective for email-thread summarization in their experiments. This work is directly relevant to the **summarization component of IRIS**, although IRIS focuses primarily on summarizing individual emails rather than complete email threads.

2.2.5 Ayodele, Khusainov and Ndzi (2007) — Email Classification and Summarization: A Machine Learning Approach

Ayodele, Khusainov, and Ndzi proposed a system that combines **email classification and summarization**. The system uses the **subject and content of email messages** to classify emails according to users' activities and automatically generate summaries of incoming messages. The authors aimed to address problems such as email overload, difficulties in prioritization, and difficulty finding previously archived messages. This study is particularly relevant to IRIS because it demonstrates an early approach to combining **email classification and summarization within a single system**. IRIS extends this idea by using supervised priority classification with **HIGH, MEDIUM, and LOW labels**, TF-IDF features, and Multinomial Naive Bayes and Logistic Regression models.

Study	Main Focus	Dataset/Corpus	Approach	Relevance to IRIS
Zhang & Xu (2015)	Email importance	Annotated email corpus	Importance classification	Supports priority classification
Assessing E-mail Intent and Tasks (2016)	Email tasks and intentions	Enron, Avocado	Task classification	Supports task/reminder features
Alkhereyf& Rambow (2017)	Email classification	Enron, Avocado	SVM, Extra-Trees + lexical/network features	Shows alternative classification features
Choudhari et al. (2020)	Email prioritization	Email dataset	Naive Bayes	Directly related to IRIS priority classification
Ray et al. (2026)	Enterprise importance labeling	Open-source email datasets	Efficient importance labeling	Supports lightweight deployment consideration

2.4 Research Gap

The reviewed studies demonstrate that email importance, tasks, and classification can be addressed using different machine-learning and feature-based approaches. However, the studies reviewed do not directly combine all the components targeted by IRIS into a single lightweight email-analysis application.

IRIS addresses this gap by combining:

- **Email priority classification** using HIGH, MEDIUM, and LOW classes.
- **Email category classification** as an additional email-analysis task.
- **Extractive email summarization** using TF-IDF-based sentence scoring.
- Comparison of **Multinomial Naive Bayes and Logistic Regression** for classification.
- A lightweight NLP pipeline based on **preprocessing, TF-IDF, and n-grams**.
- Integration of these NLP components into a working email-analysis backend.
- Additional intelligent email-management features planned as part of the developing IRIS application.

Thus, rather than relying on computationally expensive large language models, the current IRIS implementation investigates whether **traditional, lightweight NLP and machine-learning techniques can provide useful email analysis in a locally deployable system**.

Chapter 3 – Methodology

This chapter describes the dataset, preprocessing techniques, NLP techniques, algorithms, model selection, system architecture, and workflow used to develop the IRIS email analysis system.

3.1 Dataset

A dataset of **750 curated synthetic emails** was prepared for the project. The emails represent a single user's mailbox and contain realistic academic, professional, personal, financial, travel, promotional, and other email scenarios.

Each email contains the following fields:

- Email ID
- Sender
- Receiver
- Subject
- Body
- Category
- Priority
- Source

The priority dataset contains three classes:

Priority	Number of Emails
HIGH	250
MEDIUM	250
LOW	250
Total	750

For priority and category classification, the dataset was divided using an **80:20 stratified split**:

- Training data: **600 emails**
- Testing data: **150 emails**
- The test set contains 50 emails from each priority class.

An additional set of **20 emails with manually written reference summaries** was used to evaluate the summarization component using ROUGE metrics.

The Enron email dataset was also explored as a supplementary real-world email corpus. However, it was not used as the main supervised priority dataset because suitable priority labels were not available for the project.

3.2 Data Preprocessing

Email text contains unnecessary elements such as HTML tags, URLs, inconsistent capitalization, and common words. Therefore, preprocessing was performed before feature extraction and classification.

The following steps were used:

3.2.1 Lowercasing

All text was converted to lowercase so that words such as "Meeting" and "meeting" were treated as the same word.

3.2.2 HTML and URL Removal

HTML tags and URLs were removed because they generally do not provide useful information for priority classification.

3.2.3 Tokenization

Emails were divided into individual words or tokens.

3.2.4 Stop-word Removal

Common words that provide limited classification information were removed.

However, some words were deliberately retained, such as **not, no, at, before, after, until, from, to, AM, and PM**, because they can provide important meaning related to time, deadlines, and negation.

3.2.5 Lemmatization

Lemmatization was used to convert words into their meaningful base forms.

For example:

- *running* → *run*
- *meetings* → *meeting*

Why was lemmatization used?

Lemmatization helps reduce different forms of the same word while generally preserving its meaningful linguistic form. It was selected instead of using both stemming and lemmatization because applying both is unnecessary for this project.

The processed text was stored in the `clean_text` field.

3.3 NLP Techniques

3.3.1 TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) was used to convert email text into numerical features that machine-learning models can process.

TF-IDF gives higher importance to words that are frequent in a particular email but less common across the entire dataset.

Why was TF-IDF selected?

TF-IDF was selected because:

- It is suitable for text classification.
- It works well with traditional machine-learning algorithms.
- It is computationally lightweight.
- It provides interpretable text features.
- It does not require downloading a large pretrained language model.

For priority classification, the resulting TF-IDF representation contained **3,367 features**.

3.3.2 Unigrams and Bigrams

Both **unigrams** and **bigrams** were used.

- **Unigram:** one word, e.g. *deadline*
- **Bigram:** two consecutive words, e.g. *project deadline*

The vectorizer used:

- `max_features = 5000`
- `ngram_range = (1,2)`
- `min_df = 2`

Why were unigrams and bigrams selected?

A single word may not always provide enough context. Two-word combinations can capture meaningful email phrases such as:

- *interview confirmation*
- *project deadline*
- *meeting reminder*

Therefore, using both unigrams and bigrams provides more contextual information than using individual words alone.

3.3.3 Extractive Summarization

For email summarization, a lightweight **extractive summarization** approach was used.

The email is divided into sentences and TF-IDF is used to calculate the importance of each sentence. The highest-scoring sentences are selected to form the summary.

The system currently selects up to **two important sentences**.

Why was extractive summarization selected?

It was selected because:

- It is simple and suitable for the project.
- It preserves sentences from the original email.
- It can run locally.
- It does not require a large pretrained language model.
- It is computationally lighter for the project's current environment.

3.4 Algorithms and Models

Two machine-learning algorithms were implemented and compared for classification:

3.4.1 Multinomial Naive Bayes

Multinomial Naive Bayes was used because it is commonly suitable for text classification and works efficiently with TF-IDF-based features.

It was used for:

- Priority classification
- Category classification

3.4.2 Logistic Regression

Logistic Regression was used as a second classification approach.

It was selected because it performs well with high-dimensional numerical text features and provides a useful comparison with Naive Bayes.

It was also used for:

- Priority classification
- Category classification

Why were two models compared?

Instead of selecting a model without testing, both models were trained and evaluated on the same test data.

This allows the final model to be selected based on **experimental results**, using metrics such as:

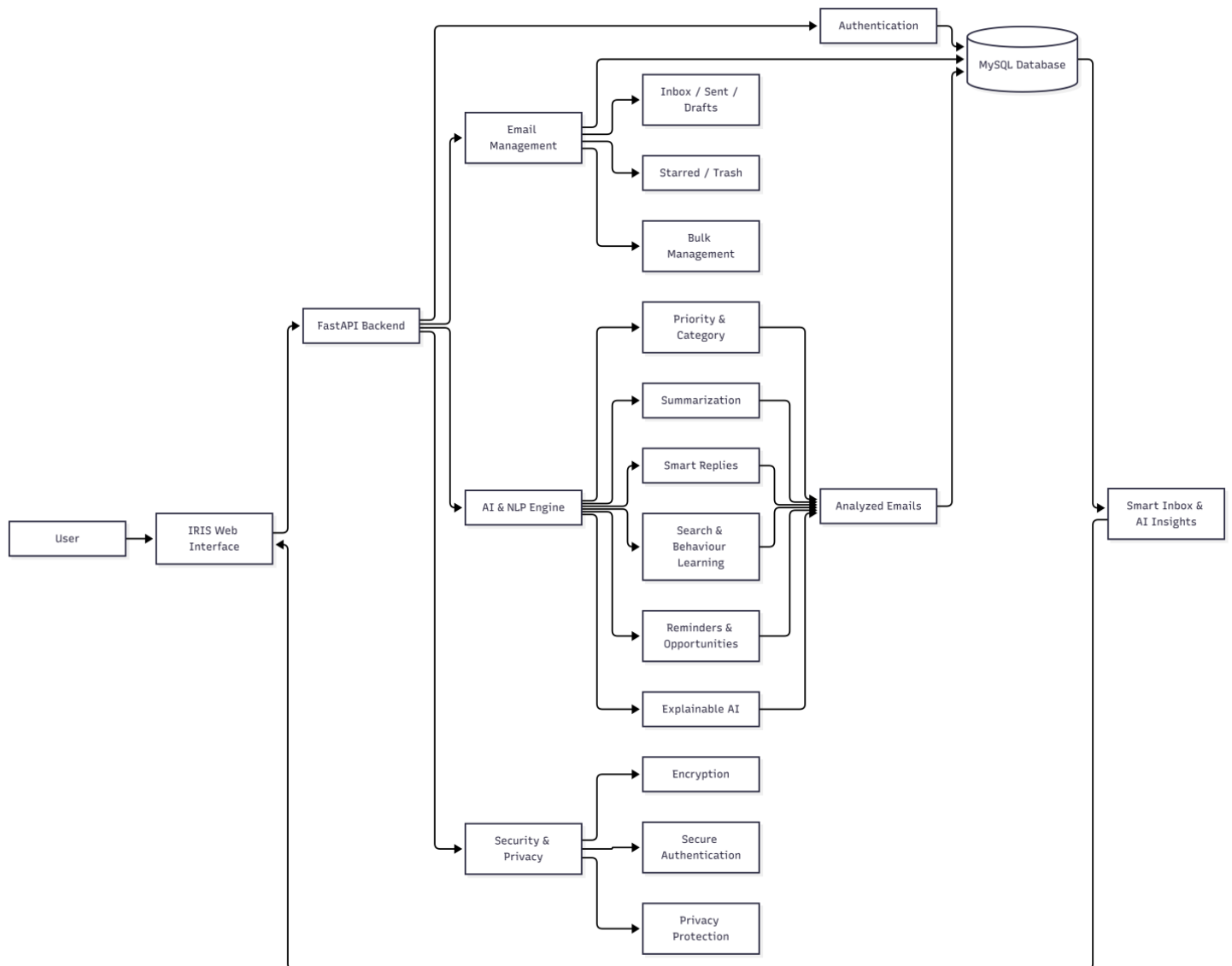
- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

The results showed:

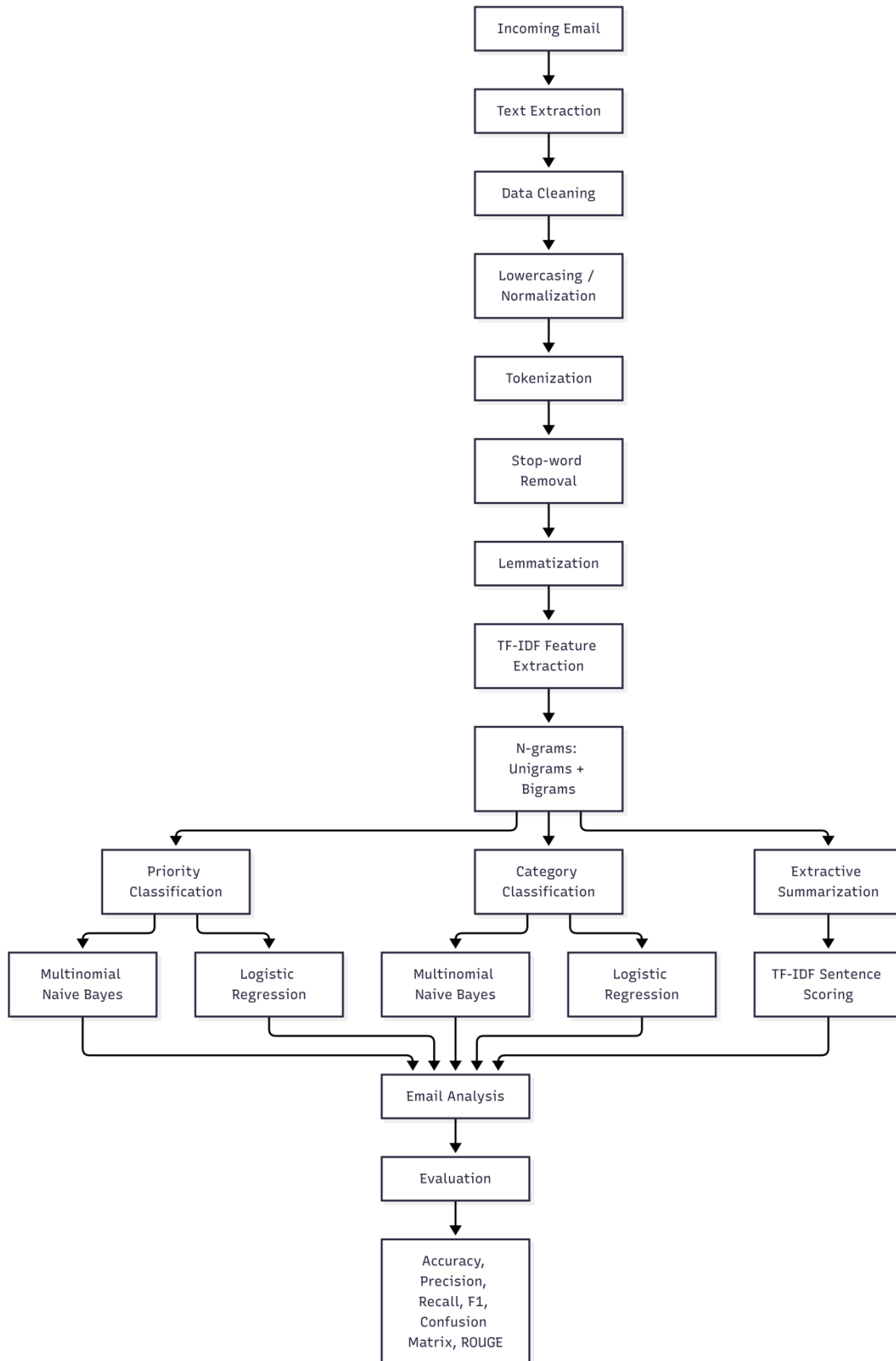
Task	Naive Bayes	Logistic Regression	Model Used
Priority Classification	94.00%	93.33%	Naive Bayes
Category Classification	60.00%	72.67%	Logistic Regression

Therefore, **Naive Bayes was used for the final priority prediction pipeline**, while **Logistic Regression was used for the category prediction pipeline**, based on their performance on the project's test data.

3.5 System Architecture



The NLP processing pipeline is:



Chapter 4 – Implementation

This chapter describes the software and hardware requirements, technologies and libraries used, major modules, important implementation code, and screenshots of the working IRIS system.

4.1 Software and Hardware Requirements

4.1.1 Software Requirements

The following software and development tools were used:

Software/Tool	Purpose
Python	Main programming language
Spyder	Python development environment
Anaconda	Python environment and package management
MySQL	Database management
FastAPI	Backend API development
Git	Version control
GitHub	Project repository and source-code management
Windows	Development platform

4.1.2 Hardware Requirements

IRIS was developed and tested on a personal computer capable of running Python, MySQL, and the FastAPI backend.

The system does not require dedicated GPU hardware because the implemented NLP models are lightweight and run on the CPU.

4.2 Libraries and Technologies Used

The main libraries and technologies used in the project are:

Library/Technology	Purpose
Pandas	Dataset loading and manipulation
NumPy	Numerical operations
NLTK	Tokenization, stop-word processing and lemmatization
Scikit-learn	TF-IDF, Naive Bayes, Logistic Regression and evaluation
Matplotlib	Graphs and result visualization
Joblib	Saving and loading trained models
FastAPI	Backend API
SQLAlchemy	Database interaction
PyMySQL	MySQL connectivity
python-dotenv	Environment variable management
python-jose	JWT authentication
Passlib/Bcrypt	Password hashing
ROUGE	Summarization evaluation
MySQL	Storage of users and emails

4.3 Main Modules

The IRIS implementation is divided into several modules.

4.3.1 Dataset and Preprocessing Module

This module:

- Loads the email dataset.
- Cleans the email text.
- Converts text to lowercase.
- Removes HTML and URLs.
- Performs tokenization.
- Removes selected stop words.
- Performs lemmatization.
- Creates the `clean_text` field.

4.3.2 TF-IDF Feature Extraction Module

This module converts processed email text into numerical TF-IDF features.

Both **unigrams** and **bigrams** are used so that the system can capture individual words as well as meaningful word combinations.

4.3.3 Priority Classification Module

This module predicts the priority of an email as:

- **HIGH**
- **MEDIUM**
- **LOW**

Multinomial Naive Bayes and Logistic Regression were compared during experimentation. The trained Naive Bayes model was saved and used for the final priority prediction pipeline.

4.3.4 Category Classification Module

This module assigns an email to a category such as:

- Academic
- Job/Internship
- Work/Project
- Meeting/Appointment
- Personal
- Travel
- Technology

- Promotion
- Newsletter
- Financial

and other categories present in the dataset.

Logistic Regression was selected for the final category prediction pipeline based on the experimental results.

4.3.5 Email Summarization Module

The summarization module uses an extractive TF-IDF-based approach.

It:

1. Divides the email into sentences.
2. Calculates TF-IDF scores for the sentences.
3. Ranks the sentences according to their scores.
4. Selects the most important sentences.
5. Generates a short summary.

4.3.6 Backend and Authentication Module

The FastAPI backend provides APIs for:

- User registration
- User login
- Authentication
- Sending emails
- Receiving emails
- Inbox management
- Reading emails
- AI-based email analysis
- Email summarization

JWT authentication is used to protect authenticated operations.

4.3.7 Database Module

MySQL is used to store application data.

The main database entities include:

- Users
- Emails

Email records can contain:

- Sender
- Receiver
- Subject
- Body
- Priority
- Category
- Summary
- Read/unread status
- Starred status
- Creation time

4.4 Important Code Snippets

4.4.1 Text Preprocessing

The preprocessing module performs basic cleaning and linguistic processing before the text is passed to the machine-learning model.

```

def preprocess_text(text):

    # Convert to string
    text = str(text)

    # Lowercase
    text = text.lower()

    # Remove HTML tags
    text = re.sub(r"<.*?>", " ", text)

    # Remove URLs
    text = re.sub(r"http\S+|www\S+", " ", text)

    # Keep: letters , numbers , spaces , colon (10:00) , slash (25/09/2026) , hyphen (25-09-2026)
    text = re.sub(r"[^a-zA-Z0-9\s:/~]", " ", text)

    # Remove extra spaces
    text = re.sub(r"\s+", " ", text).strip()

    # Tokenization
    tokens = word_tokenize(text)

    # Stopword removal
    tokens = [
        word for word in tokens
        if word not in custom_stop_words
    ]

    # Lemmatization
    tokens = [
        lemmatizer.lemmatize(word)
        for word in tokens
    ]

    # Convert tokens back to text
    cleaned_text = " ".join(tokens)

    return cleaned_text

```

This processed text is then used for feature extraction.

4.4.2 TF-IDF Feature Extraction

```
tfidf = TfidfVectorizer(  
    max_features=5000,  
    ngram_range=(1, 2),  
    min_df=2  
)  
  
# -----  
# 6. Fit TF-IDF on training data  
# -----  
  
X_train_tfidf = tfidf.fit_transform(X_train)  
  
# -----  
# 7. Transform test data  
# -----  
  
X_test_tfidf = tfidf.transform(X_test)
```

The vectorizer is fitted only on the training data and then used to transform the test data.

4.4.3 Priority Classification

The final priority model uses Multinomial Naive Bayes.

```
model = MultinomialNB()  
  
model.fit(X_train_tfidf, y_train)  
  
# -----  
# 5. Save model  
# -----  
  
model_path = r"E:\IRIS\models\priority_nb_model.pkl"  
vectorizer_path = r"E:\IRIS\models\priority_tfidf_vectorizer.pkl"
```

The trained model and TF-IDF vectorizer are saved using Joblib so that they can later be used to predict the priority of new emails.

4.4.4 Category Classification

Logistic Regression was used for the final category prediction.

```
lr_model = LogisticRegression(  
    max_iter=1000,  
    random_state=42  
)  
  
lr_model.fit(X_train_tfidf, y_train)  
  
lr_predictions = lr_model.predict(X_test_tfidf)  
  
lr_accuracy = accuracy_score(y_test, lr_predictions)
```

4.4.5 Extractive Summarization

The summarization module calculates TF-IDF scores for individual sentences.

```
def summarize_email(subject, body, max_sentences=2):  
    text = f"{subject}. {body}"  
    sentences = split_sentences(text)  
  
    if len(sentences) <= max_sentences:  
        return " ".join(sentences)  
  
    vectorizer = TfidfVectorizer(  
        stop_words="english"  
    )  
  
    matrix = vectorizer.fit_transform(sentences)  
    scores = np.asarray(matrix.sum(axis=1)).ravel()  
    ranked_indices = np.argsort(scores)[::-1][:max_sentences]  
    selected_indices = sorted(ranked_indices)  
    summary = " ".join(  
        sentences[i] for i in selected_indices  
    )  
  
    return summary
```

The selected sentences are arranged in their original order to produce the final summary.

4.4.6 Automatic Email Analysis

When a new email is sent through the backend, the system automatically performs all three main NLP tasks.

```
priority = predict_priority(
    email_data.subject,
    email_data.body
)

category = predict_category(
    email_data.subject,
    email_data.body
)
summary = generate_summary(
    email_data.subject,
    email_data.body
)

new_email = Email(
    sender_id=current_user.id,
    receiver_id=receiver.id,
    subject=email_data.subject,
    body=email_data.body,
    priority=priority,
    category=category,
    summary=summary
)

db.add(new_email)
db.commit()
db.refresh(new_email)

return {
    "message": "Email sent successfully",
    "email_id": new_email.id,
    "sender": current_user.email,
    "receiver": receiver.email,
    "subject": new_email.subject,
    "priority": new_email.priority,
    "category": new_email.category
}
```

4.5 Screenshots of Implementation.

Figure 4.1: FastAPI Swagger interface showing IRIS backend APIs.

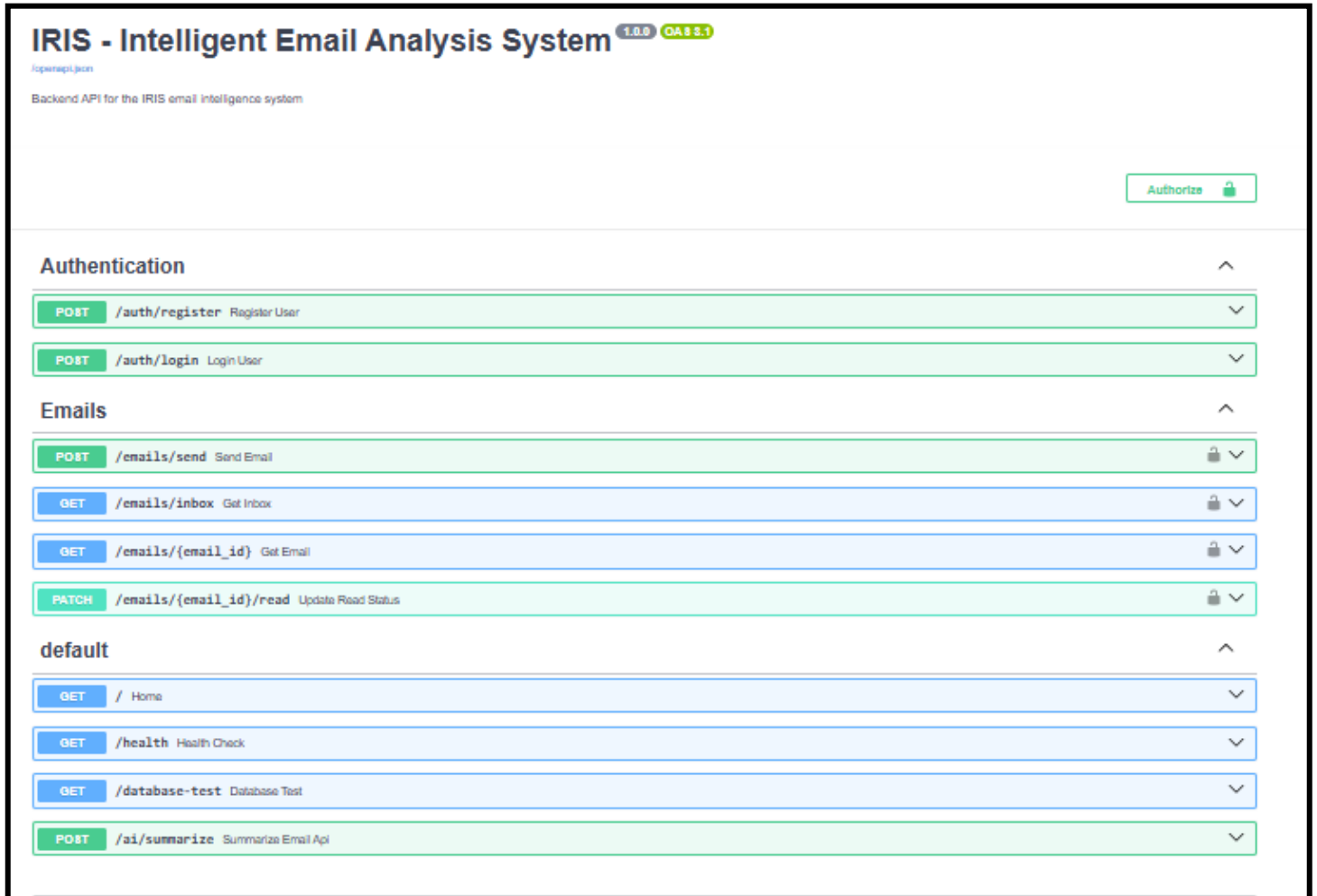


Figure 4.2: User Login

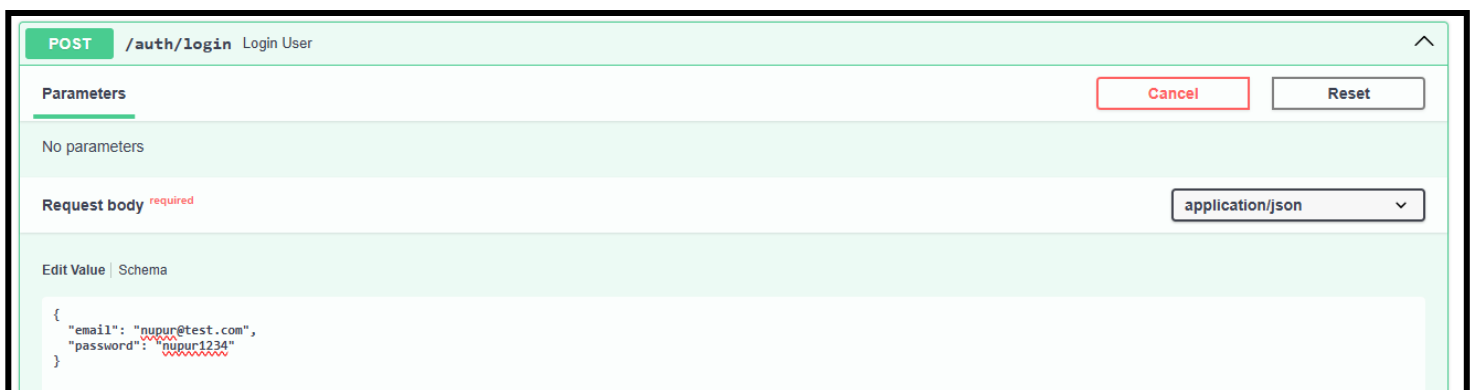


Figure 4.3: Authentication using JWT

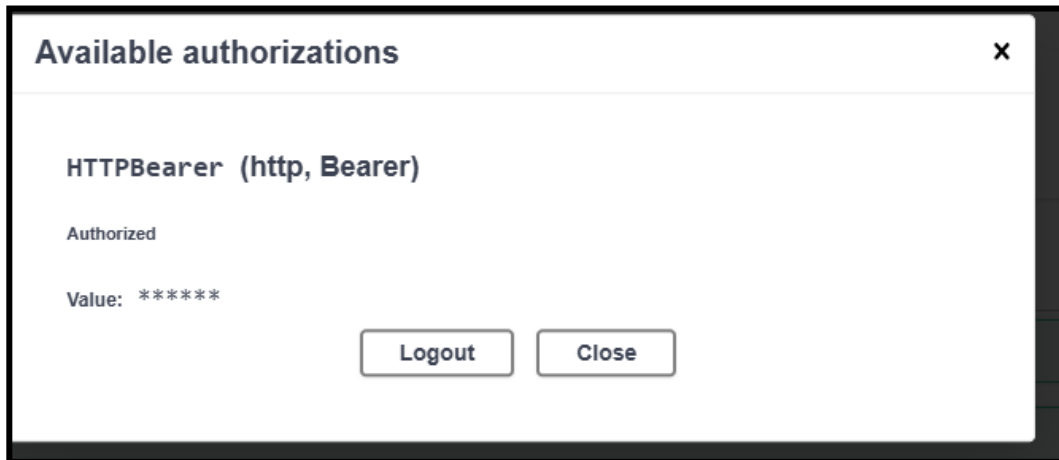


Figure 4.4: Sending email to another user



Figure 4.5: Email Sent

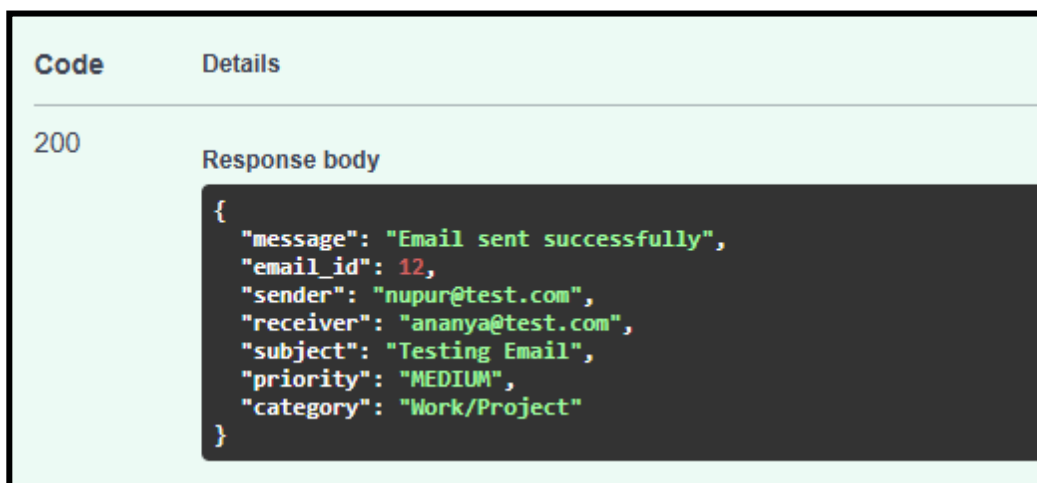


Figure 4.6: Email Received by user

```
{
  "user": "ananya@test.com",
  "count": 5,
  "emails": [
    {
      "id": 12,
      "sender_id": 1,
      "subject": "Testing Email",
      "body": "Hey , Ananya , this is a testing email to check the functionality of IRIS",
      "priority": "MEDIUM",
      "category": "Work/Project",
      "summary": "Testing Email. Hey , Ananya , this is a testing email to check the functionality of IRIS",
      "is_read": false,
      "is_starred": false,
      "created_at": "2026-09-24T22:36:23"
    },
    {
```

Figure 4.7: Summarization, Priority and Category shown by the system to the user

```

    "id": 9,
    "sender_id": 1,
    "subject": "Interview Schedule Change",
    "body": "Hi Nupur, I hope you had a good weekend. The weather has been quite pleasant lately. I wanted to share a few updates regarding our project. We have completed the database design and frontend prototype. The backend integration is still pending. Our project review has been moved to Friday at 2 PM. Please prepare the system demonstration and bring the latest presentation. The review will be conducted in Lab 3. Let me know if you have any questions. Regards, Project Coordinator",
    "priority": "HIGH",
    "category": null,
    "summary": "We have completed the database design and frontend prototype. Please prepare the system demonstration and bring the latest presentation.",
    "is_read": false,
    "is_starred": false,
    "created_at": "2026-09-23T11:47:34"
  },
  {

```

Chapter 5 – Results and Discussion

This chapter presents the experimental results obtained from the priority classification, category classification, and email summarization components of IRIS. The results are evaluated using appropriate performance metrics and are supported by graphs, confusion matrices, and error analysis.

5.1 Experimental Setup

The classification experiments were performed using the curated dataset of **750 emails**.

For classification:

- **600 emails** were used for training.
- **150 emails** were used for testing.
- Stratified splitting was used so that the test set contained 50 emails from each priority class.
- TF-IDF with unigrams and bigrams was used for feature extraction.
- Multinomial Naive Bayes and Logistic Regression were compared.

For summarization:

- **20 emails** were evaluated.
- Each email had a manually written reference summary.
- ROUGE-1, ROUGE-2, and ROUGE-L were used for evaluation.

5.2 Priority Classification Results

The main classification task of IRIS is to determine whether an email is **HIGH, MEDIUM, or LOW priority**.

Two models were compared.

Model	Accuracy	Macro F1-score
Multinomial Naive Bayes	94.00%	0.94
Logistic Regression	93.33%	0.93

Multinomial Naive Bayes achieved an accuracy of **94.00%**, correctly classifying **141 out of 150 test emails**.

Dataset loaded successfully!
Total emails: 750

TF-IDF shape:
(600, 3367)

Training Multinomial Naive Bayes...
Model training completed!

Accuracy:
0.9400

Classification Report:

	precision	recall	f1-score	support
HIGH	0.96	1.00	0.98	50
MEDIUM	0.94	0.88	0.91	50
LOW	0.92	0.94	0.93	50
accuracy			0.94	150
macro avg	0.94	0.94	0.94	150
weighted avg	0.94	0.94	0.94	150

Confusion Matrix:
[[50 0 0]
[2 44 4]
[0 3 47]]

Dataset loaded successfully!
Total emails: 750

Training Logistic Regression...
Model training completed!

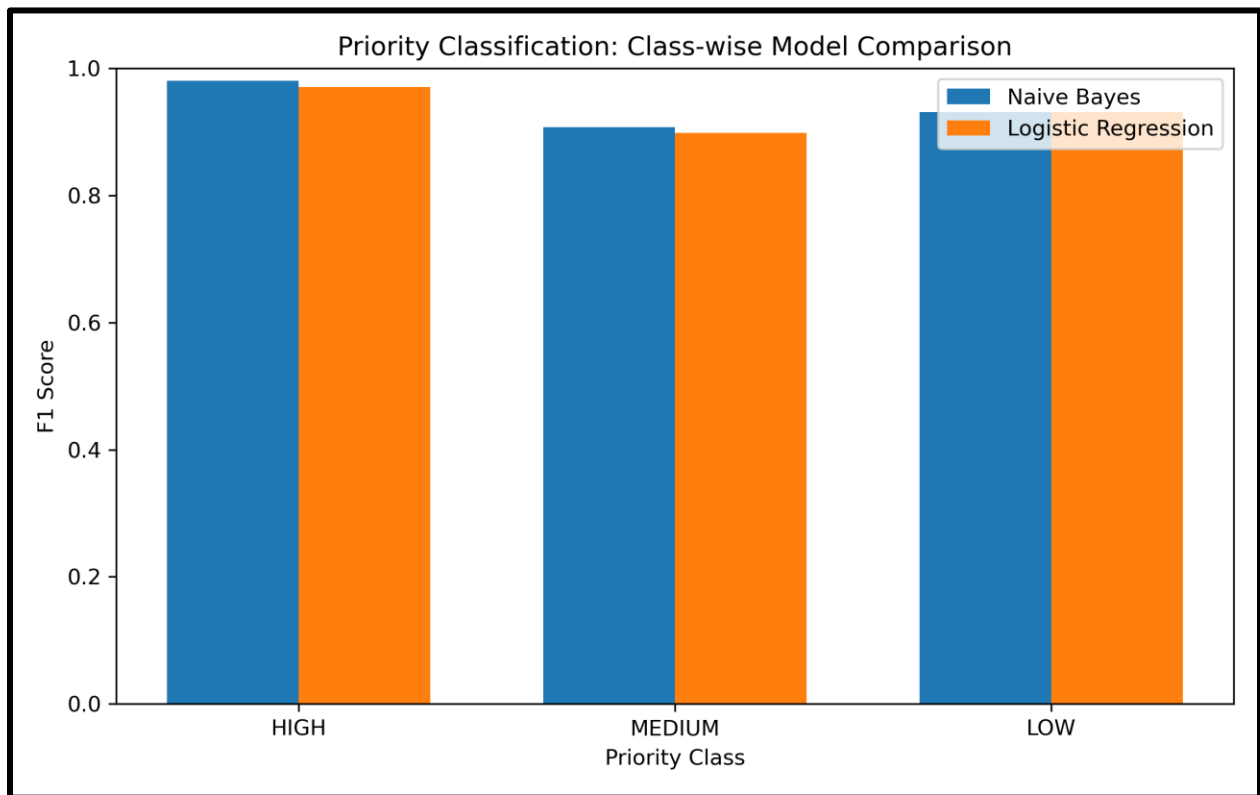
Accuracy:
0.9333

Classification Report:

	precision	recall	f1-score	support
HIGH	0.96	0.98	0.97	50
MEDIUM	0.92	0.88	0.90	50
LOW	0.92	0.94	0.93	50
accuracy			0.93	150
macro avg	0.93	0.93	0.93	150
weighted avg	0.93	0.93	0.93	150

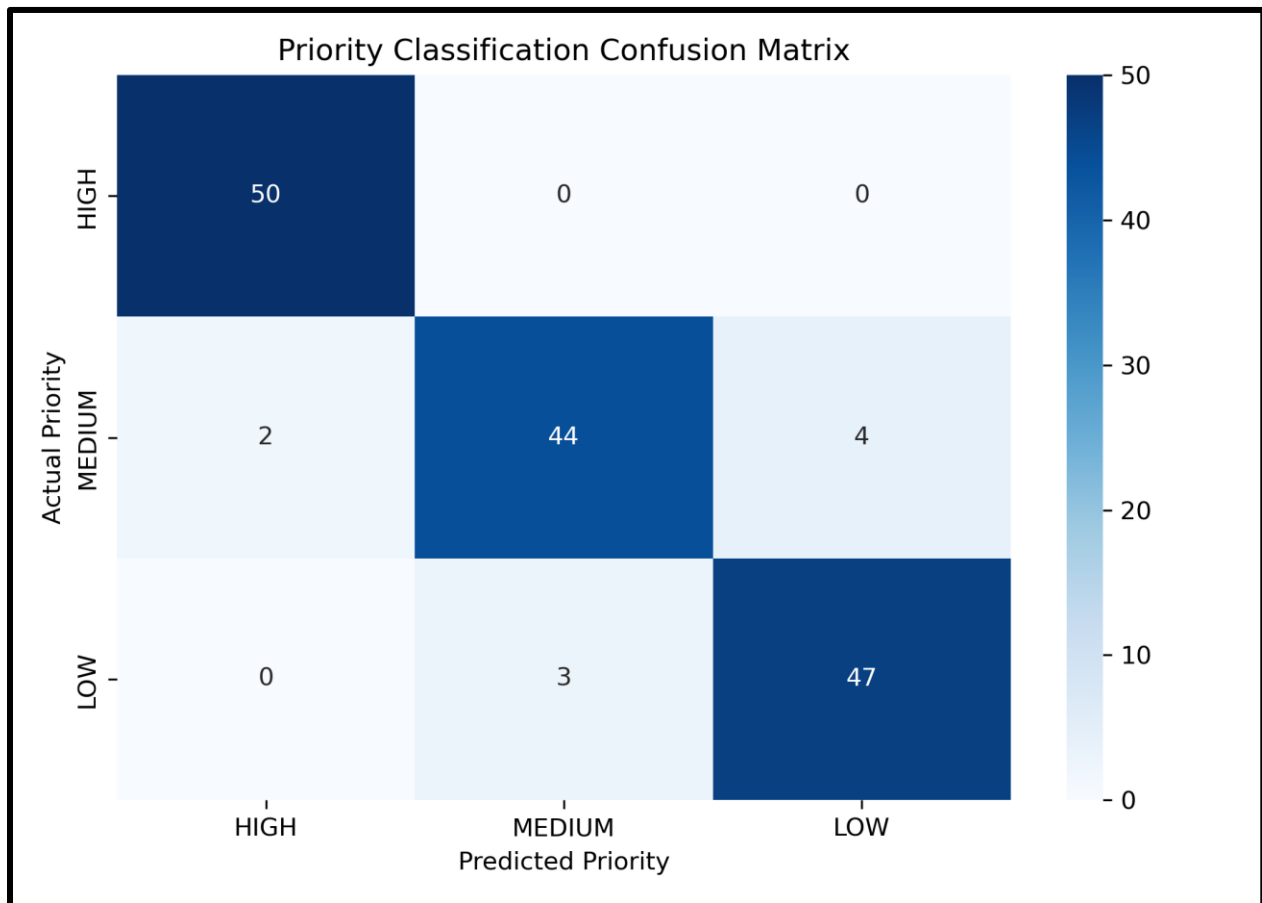
Confusion Matrix:
[[49 1 0]
[2 44 4]
[0 3 47]]

Figure 5.1 – Priority Model Comparison



The two models produced similar results, but Multinomial Naive Bayes achieved slightly higher overall accuracy and macro F1-score. This result supports its selection for the priority classification component.

Figure 5.2 – Priority Classification Confusion Matrix



Out of 150 test emails:

- **50 HIGH emails were correctly classified.**
- **44 MEDIUM emails were correctly classified.**
- **47 LOW emails were correctly classified.**

The main errors were:

- **3 LOW → MEDIUM**
- **2 MEDIUM → HIGH**
- **4 MEDIUM → LOW**

This indicates that the model had more difficulty distinguishing between MEDIUM and LOW emails than identifying HIGH-priority emails.

5.3 Priority Classification Error Analysis

Nine incorrect predictions were observed in the test set.

Examples include:

Email	Actual	Predicted
Thank You for Applying	MEDIUM	HIGH
New Course Added to the Learning Platform	MEDIUM	LOW
New Terms and Conditions Notice	LOW	MEDIUM
General Update on the Garden Project	LOW	MEDIUM
New Features Added to the App	MEDIUM	LOW
New Mobile Banking Feature Available	MEDIUM	LOW

The errors mainly occurred when emails contained neutral or informational language that could be associated with more than one priority level.

```
Total test emails: 150
Correct predictions: 141
Incorrect predictions: 9

ERROR ANALYSIS:
email_id          subject priority predicted_priority
M001      Thank You for Applying  MEDIUM      HIGH
M160 New Course Added to the Learning Platform  MEDIUM      LOW
L243      New Terms and Conditions Notice      LOW      MEDIUM
L222      General Update on the Garden Project  LOW      MEDIUM
M081      New Features Added to the App        MEDIUM      LOW
M084      New Mobile Banking Feature Available  MEDIUM      LOW
M008      Internship Certificate Request Received  MEDIUM      HIGH
M219      Storage Usage Update                  MEDIUM      LOW
L244      Your Annual Policy Summary            LOW      MEDIUM

CONFUSION BY ACTUAL AND PREDICTED PRIORITY:
priority predicted_priority
LOW      MEDIUM          3
MEDIUM   HIGH            2
         LOW            4
dtype: int64
```

5.4 Category Classification Results

Category classification was implemented as an additional email-analysis component.

Two models were compared:

Model	Accuracy	Macro F1-score	Weighted F1-score
Multinomial Naive Bayes	60.00%	0.50	0.56
Logistic Regression	72.67%	0.63	0.69

Logistic Regression achieved the higher accuracy of **72.67%**, correctly classifying **109 out of 150 test emails**.

Therefore, Logistic Regression was selected for the final category classification pipeline.

Figure 5.3 – Category Class Distribution

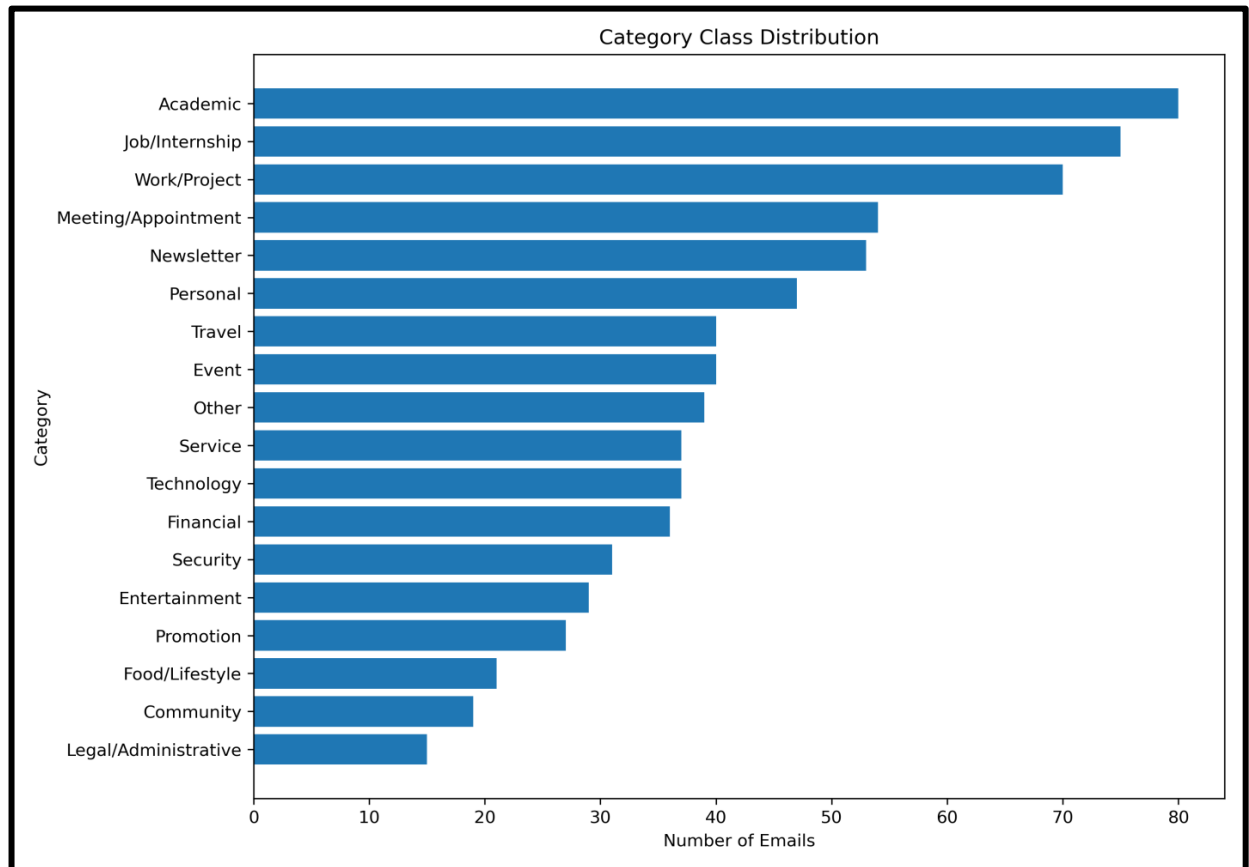
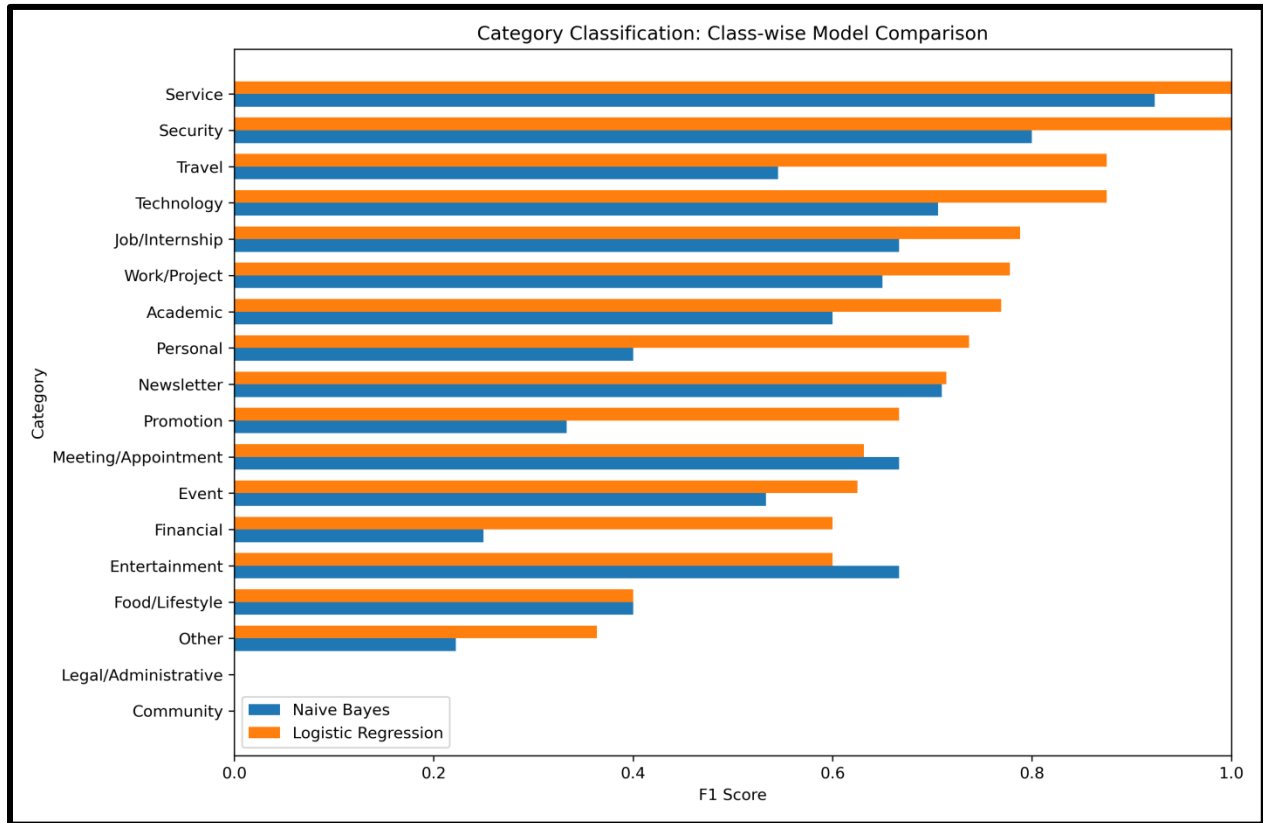


Figure 5.3 – Category Model Comparison



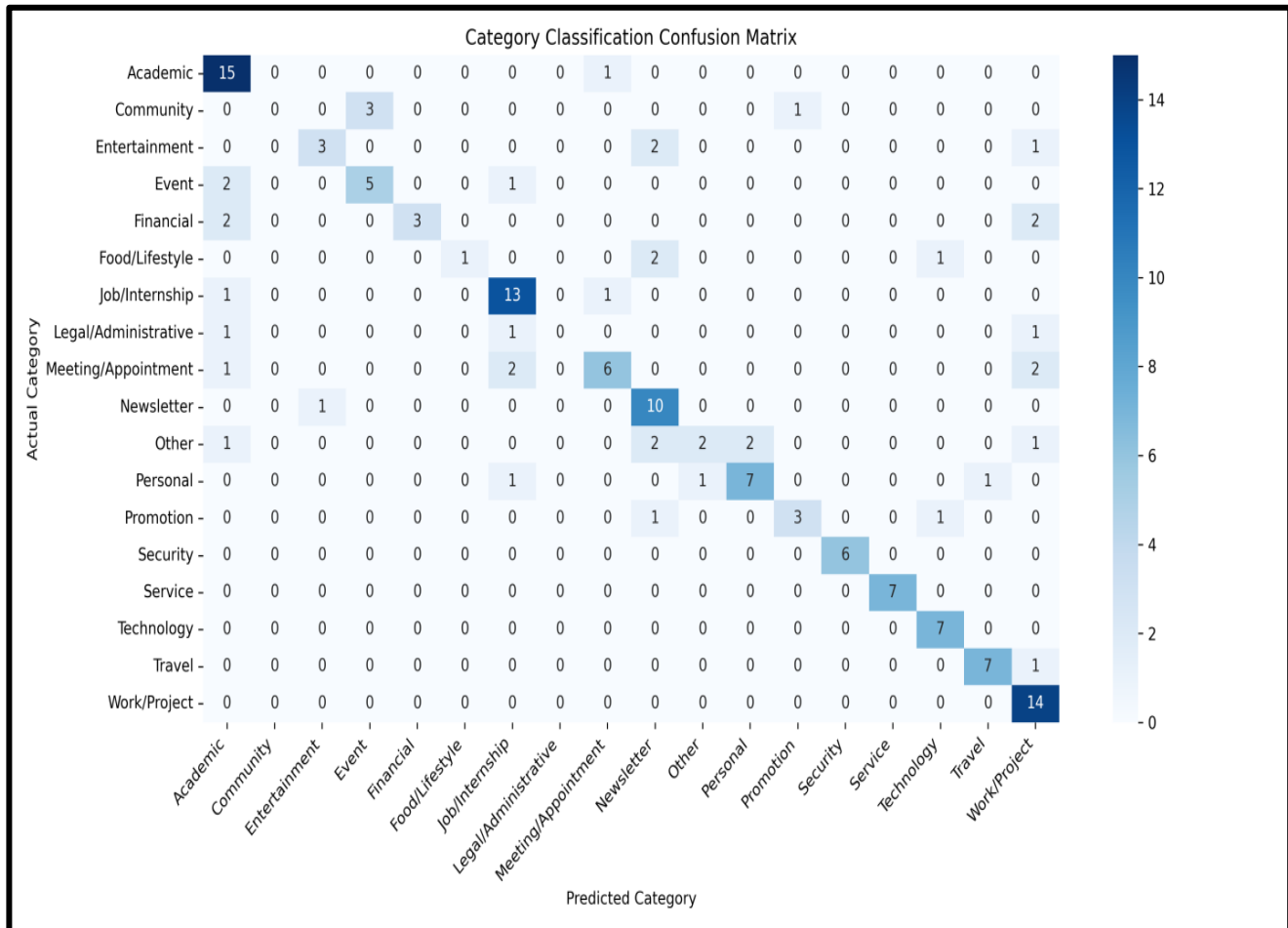
Category classification is more challenging because some categories have similar vocabulary and contextual meaning. For example, emails related to meetings, work, academic activities, and job-related activities can contain overlapping terms.

Logistic Regression performed better than Naive Bayes on the test set and was therefore selected for category prediction.

5.6 Category Classification Confusion Matrix

The Logistic Regression confusion matrix was generated from the 150 test emails.

Figure 5.4 – Category Classification Confusion Matrix



Logistic Regression correctly classified 109 of the 150 test emails, while 41 were incorrectly classified.

The errors were distributed across several categories. Some categories had fewer examples in the test set, which can make their individual precision and recall values less stable.

5.7 Category Classification Error Analysis

The Logistic Regression model was selected for category classification because it achieved **72.67% accuracy** on the test set. Out of 150 test emails, **109 were correctly classified and 41 were incorrectly classified.**

The errors were mainly caused by overlap between categories with similar topics or contexts.

Examples of Misclassified Emails

Email Subject	Actual Category	Predicted Category
Power Backup Failing During Exam Week	Other	Academic
A Short Look Back	Other	Newsletter
Volunteers Needed for Sports Day	Event	Academic
Annual Policy Review Suggestion	Financial	Work/Project
Need to Move Our Call	Meeting/Appointment	Work/Project
Passport Renewal Reminder	Legal/Administrative	Job/Internship
Auto-Debit Failed	Financial	Academic
Uncle's Flight Details	Personal	Travel
Borrowing Your Extension Cord	Personal	Job/Internship
Upcoming Webinar	Event	Job/Internship
Behind the Scenes This Week	Newsletter	Entertainment
Meeting Slot Options for Next Week	Academic	Meeting/Appointment

Main Error Patterns

The confusion matrix showed several recurring patterns:

- **Community** → **Event/Promotion**
- **Entertainment** → **Newsletter/Work/Project**
- **Financial** → **Academic/Work/Project**

- Food/Lifestyle → Newsletter/Technology
- Legal/Administrative → Academic/Job/Internship/Work
- Meeting/Appointment → Academic/Job/Internship/Work
- Other → Academic/Newsletter/Personal/Work
- Personal → Job/Internship/Other/Travel
- Promotion → Newsletter/Technology
- Travel → Work/Project

The category classification task was more difficult than priority classification because the system had to distinguish between 18 different categories. Some emails contain vocabulary that can reasonably belong to multiple categories.

For example, an email about an **exam-week power problem** may contain academic-related words, even though its original category is *Other*. Similarly, a **flight-related email** can be classified as *Travel* even when it is personally sent by a family member.

Another source of difficulty is the relatively small number of examples available for some categories. Categories with fewer test examples can have less stable performance than categories with more training examples.

The error analysis therefore shows that the model's mistakes were not concentrated in only one category; they occurred mainly where the **subject and context of an email overlap across categories**.

```
In [2]: %runfile E:/IRIS/src/category_error_analysis.py --wdir
```

Total test emails: 150
Correct predictions: 109
Incorrect predictions: 41

ERROR ANALYSIS:

email_id	subject	category	predicted_category
H099	Power Backup Failing During Exam Week	Other	Academic
L249	A Short Look Back	Other	Newsletter
M205	Volunteers Needed for Sports Day	Event	Academic
M218	Annual Policy Review Suggestion	Financial	Work/Project
H073	Need to Move Our Call - Something Came Up	Meeting/Appointment	Work/Project
H094	Passport Renewal - Document Verification Pending	Legal/Administrative	Job/Internship
H171	Auto-Debit Failed - Retry Before Tonight	Financial	Academic
M064	Tree Trimming Completed	Personal	Other
H224	Uncle's Flight Got Cancelled, Need Help Rebooking	Personal	Travel
H250	Neighbor's Wall Collapsed, Need Help Today	Other	Work/Project
M199	Borrowing Your Extension Cord	Personal	Job/Internship
L067	Cafes Worth Trying This Month	Food/Lifestyle	Newsletter
M204	Upcoming Webinar Series Announced	Event	Job/Internship
L221	Recap From the Music Society Jam	Community	Event
L144	Behind the Scenes This Week	Newsletter	Entertainment
M072	Annual Sports Day Volunteers Needed	Event	Academic
H211	Final Interview Slot - Confirm by Evening	Meeting/Appointment	Job/Internship
L017	Reader Recommendations Based on Your History	Promotion	Newsletter
H069	Panel Waiting - Please Join Now	Meeting/Appointment	Job/Internship
L060	Fan Theories Roundup	Entertainment	Newsletter
H100	Pet Gone Missing Near the Park	Other	Personal
H206	Layover Changed - Need to Confirm Pickup	Meeting/Appointment	Work/Project
M108	Mentor Matching Update	Job/Internship	Academic
L099	A Look Back at Our Year	Other	Newsletter
L125	Customer Photos From This Season	Community	Promotion

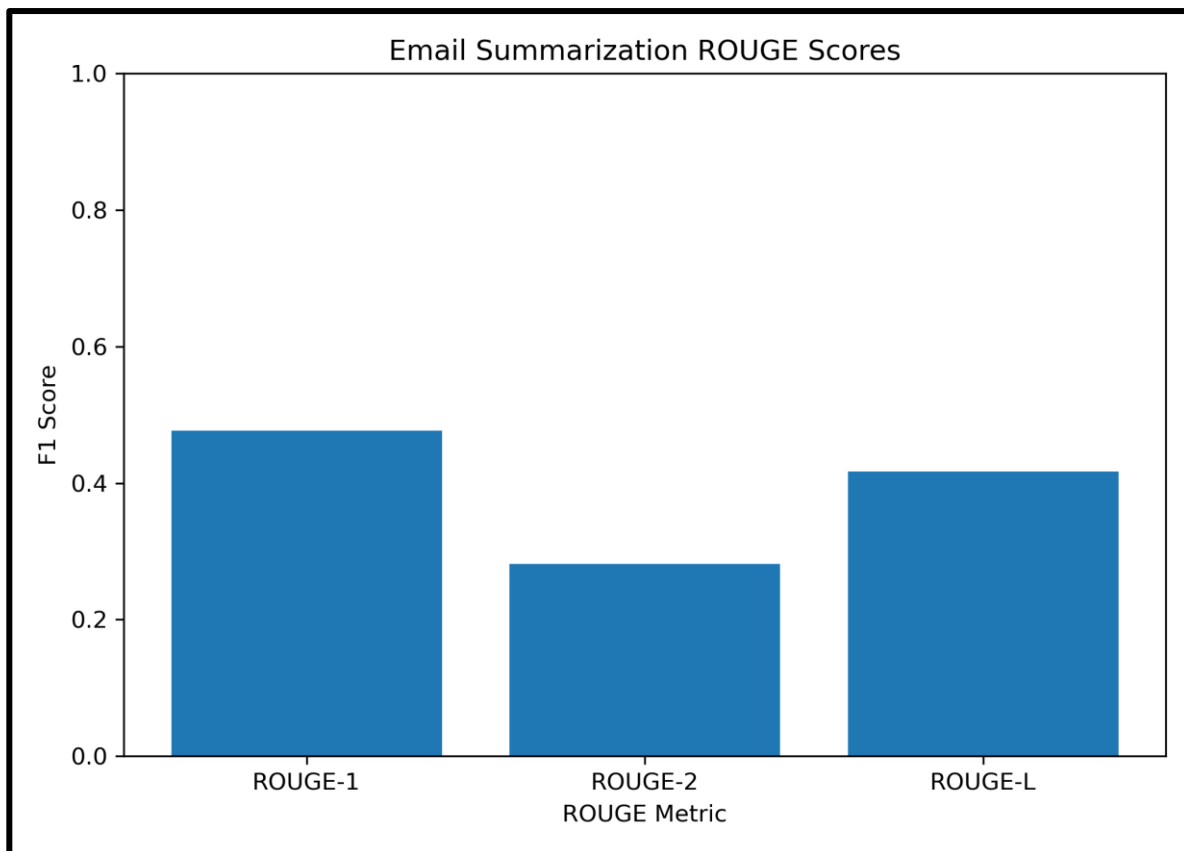
5.7 Email Summarization Results

The summarization component was evaluated using **20 emails** with manually prepared reference summaries.

The following ROUGE scores were obtained:

```
===== SUMMARIZATION EVALUATION =====  
ROUGE-1: 0.4770  
ROUGE-2: 0.2818  
ROUGE-L: 0.4172  
  
Detailed results saved to:  
E:\IRIS\results\summarization_metrics.csv
```

Figure 5.5 – Summarization ROUGE Scores



ROUGE-1 measures overlap of individual words between the generated and reference summaries, while ROUGE-2 considers two-word sequences and ROUGE-L considers the longest matching sequence.

The ROUGE-1 score of **0.4770** indicates substantial word-level overlap on the evaluation set. The lower ROUGE-2 score of **0.2818** indicates that exact two-word sequence matching was more limited.

These results represent the performance of the implemented extractive summarization approach on the prepared 20-email evaluation set.

5.8 Overall Results

The major experimental results of IRIS are summarized below:

Component	Model/Method	Result
Priority Classification	Multinomial Naive Bayes	94.00% Accuracy
Priority Classification	Logistic Regression	93.33% Accuracy
Category Classification	Multinomial Naive Bayes	60.00% Accuracy
Category Classification	Logistic Regression	72.67% Accuracy
Summarization	TF-IDF Extractive	ROUGE-1: 0.4770
		ROUGE-2: 0.2818
		ROUGE-L: 0.4172

The results demonstrate that the implemented NLP pipeline can perform the core tasks of IRIS: **priority classification, category classification, and email summarization.**

One important limitation is that the supervised classification dataset is a **curated synthetic dataset** rather than a large real-world labelled email dataset. Therefore, the reported classification results describe performance on this project's test data and should not be interpreted as universal performance on all email datasets.

Overall, the results support the feasibility of combining traditional NLP techniques with machine-learning models to build an intelligent email analysis system.

Chapter 6 – Conclusion and Future Scope

6.1 Achievements of the Project

The IRIS project successfully implements an intelligent email analysis system using Natural Language Processing and machine-learning techniques.

The major achievements of the project are:

- A dataset of **750 emails** was prepared for email classification.
- An NLP preprocessing pipeline was implemented using **lowercasing, tokenization, stop-word removal, and lemmatization**.
- **TF-IDF with unigrams and bigrams** was used for text representation.
- Two machine-learning algorithms, **Multinomial Naive Bayes and Logistic Regression**, were implemented and compared.
- Priority classification was implemented for **HIGH, MEDIUM, and LOW** emails.
- Naive Bayes achieved **94.00% accuracy** for priority classification on the test set.
- Category classification was implemented as an additional email-analysis feature.
- Logistic Regression achieved **72.67% accuracy** for category classification on the test set.
- A lightweight **extractive email summarization** method was implemented.
- Summarization was evaluated using **ROUGE-1, ROUGE-2, and ROUGE-L**.
- The NLP models were integrated with a **FastAPI backend**.
- **MySQL** was used for storing users and email information.
- Authentication and email-management functionality were implemented.
- The NLP system automatically generates the **priority, category, and summary** when an email is processed.

6.2 Limitations

Although the project successfully demonstrates the required NLP tasks, it has some limitations.

6.2.1 Dataset Limitation

The main classification dataset consists of **curated synthetic emails** rather than a large real-world labelled email dataset. Therefore, the obtained accuracy represents performance on the project's dataset and may differ when applied to real-world email data.

6.2.2 Category Classification

Category classification is more challenging because the system handles multiple categories and some categories have similar contexts. The current category classification accuracy was **72.67%**.

6.2.3 Summarization

The current summarization system uses an extractive TF-IDF-based approach. It selects important sentences from the original email rather than generating completely new sentences.

6.2.4 Advanced AI Models

Large transformer-based models were not used in the current implementation because the project focuses on a lightweight approach that can run locally without requiring large model downloads.

6.3 Future Scope

The IRIS system can be further improved in several ways.

6.3.1 Larger Real-World Dataset

A larger and more diverse labelled email dataset can be used to improve the generalization of the classification models.

6.3.2 Advanced Summarization

Future versions can use advanced NLP or transformer-based summarization models to generate more concise and context-aware summaries.

6.3.3 Smart Reply Generation

The system can be extended to generate suitable reply suggestions based on the content and context of an email.

6.3.4 Personalized Email Management

Future versions can learn from a user's interactions and preferences to improve priority prediction and email organization.

6.3.5 Smart Reminders

IRIS can identify dates, deadlines, appointments, interviews, and other important events and automatically create reminders.

6.3.6 Opportunity Detection

The system can identify potentially important opportunities such as internships, jobs, scholarships, workshops, and academic events.

6.3.7 Improved Privacy and Security

Future versions can implement stronger privacy-preserving techniques and more advanced encryption mechanisms for protecting sensitive email information.

6.3.8 Improved Category Classification

Additional training data and more advanced text-representation techniques can be used to improve classification of categories with overlapping meanings.

6.4 Conclusion

IRIS demonstrates how Natural Language Processing can be applied to reduce the effort required to manage emails. The system combines **text preprocessing, TF-IDF feature extraction, n-grams, machine-learning classification, and extractive summarization** into a single email analysis system.

The priority classification component achieved **94.00% accuracy** using Multinomial Naive Bayes, while the category classification component achieved **72.67% accuracy** using Logistic Regression on the project's test dataset. The summarization component was evaluated using ROUGE metrics and produced measurable results on the prepared evaluation set.

The project also connects these NLP components to a functional **FastAPI and MySQL-based backend**, allowing email analysis to be performed as part of the application workflow.

Overall, the project demonstrates the practical use of traditional NLP and machine-learning techniques for **email priority classification, category classification, and summarization**, while providing a foundation for further intelligent email-management features.

References

F. Zhang and K. Xu, “Annotation and classification of an email importance corpus,” in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pp. 651–656, Beijing, China, 2015, doi: 10.3115/v1/P15-2107

M. Sappelli, G. Pasi, S. Verberne, M. de Boer, and W. Kraaij, “Assessing e-mail intent and tasks in e-mail messages,” *Information Sciences*, vol. 358–359, pp. 1–17, 2016, doi: 10.1016/j.ins.2016.03.002.

S. Alkhereyf and O. Rambow, “Work Hard, Play Hard: Email Classification on the Avocado and Enron Corpora,” in *Proceedings of TextGraphs-11: Workshop on Graph-based Methods for Natural Language Processing*, pp. 57–65, 2017, doi: 10.18653/v1/W17-2408.

D. M. Zajic, B. J. Dorr, and J. Lin, “Single-document and multi-document summarization techniques for email threads using sentence compression,” *Information Processing & Management*, vol. 44, no. 4, pp. 1600–1610, 2008, doi: 10.1016/j.ipm.2007.09.007.

T. Ayodele, R. Khusainov, and D. Ndzi, “Email classification and summarization: A machine learning approach,” in *IET Conference on Wireless, Mobile and Sensor Networks 2007 (CCWMSN07)*, pp. 805–808, 2007, doi: 10.1049/cp:20070271.